
BIL 482 – Software Design and Architecture

FANTASY TACTICS LEAGUE

Final Design and Architecture Report

Group S.A.Z

Muhammet Emir Dizdar – 211401028

Doruk Dolaşık – 211401030

Serhan Tezcan – 211401024

April 2026

1.1 Project Purpose

Fantasy Tactics League is a sophisticated web-based simulation platform that allows users to manage virtual football squads composed of real Premier League players. The project is designed not merely as a gaming application but as a technical demonstration of Clean Architecture and GoF Design Patterns. By integrating real-world match data from the official Fantasy Premier League (FPL) API, the system provides a dynamic environment where software components must handle complex business rules, such as budget constraints, tactical role modifiers, and automated scoring engines.

Objectives

- **Modular Design:** Implementing a "Modular Monolith" architecture where use cases are isolated to allow for future microservices decomposition.
- **Pattern Implementation:** Demonstrating the effective use of at least five design patterns (Factory, Strategy, Observer, Decorator, Singleton) to solve recurring software design problems.
- **Data Integration:** Establishing a robust ETL (Extract, Transform, Load) pipeline to process JSON data from external APIs into a localised SQLite persistence layer.
- **Event-Driven Communication:** Utilising a centralised Event Bus to achieve loose coupling between core modules like the Scoring Engine and the Global Leaderboard.

1.2 Project Scope

The following table defines the boundaries of the system, distinguishing features that are included in scope from those explicitly excluded.

Feature Category	Included in Scope	Excluded from Scope
Team Management	Squad building, budget tracking (£100.0m), and transfers.	Real-time live match streaming.
Scoring	Automated position-based scoring and tactical modifiers.	Betting or real-money gambling features.
Ranking	Global leaderboard with in-memory caching for performance.	Social networking or friend-list systems.
Data	Official FPL API integration and historical stat tracking.	Mobile-native app development (iOS/Android).

1.3 Definitions, Acronyms, and Abbreviations

Definitions

Term	Definition
Fantasy Team	A virtual squad of 15 real Premier League players selected by a user.
Gameweek	A round of Premier League matches; the primary unit of scoring time.
Modular Monolith	A single-deployment application internally organised into isolated, domain-specific modules.
Event Bus	A centralised Pub/Sub mediator through which modules communicate without direct dependencies.
ETL Pipeline	Extract-Transform-Load process; fetches raw JSON from the FPL API, cleans it, and loads it into SQLite.
Singleton Cache	In-memory object instance (GlobalLeaderboard) that stores ranking data for near-instant retrieval.
Tactical Modifier	A Decorator-pattern wrapper (ModifierStrategy) that applies a score multiplier (e.g., captain $\times 2$, bench $\times 0$).
Validation Strategy	A Strategy-pattern class encapsulating one squad-rule check (budget, size, position, club limit).

Acronyms

Acronym	Full Form
FPL	Fantasy Premier League
GK	Goalkeeper
DEF	Defender
MID	Midfielder
FWD	Forward
UC	Use Case
GoF	Gang of Four
SPA	Single Page Application
OOP	Object-Oriented Programming
REST	Representational State Transfer
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CF	Coupling Factor
ICT	Influence, Creativity and Threat (FPL metric)

Abbreviations

Abbreviation	Meaning
SRS	Software Requirements Specification
SDD	Software Design Document
ERD	Entity-Relationship Diagram
DFD	Data Flow Diagram
NFR	Non-Functional Requirement
FR	Functional Requirement
RDBMS	Relational Database Management System
PH	Person-Hours
UX	User Experience
DXA	Document Exchange Architecture (Word measurement unit)

1.4 References

- [1] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- [2] Fantasy Premier League. (2024). FPL API – bootstrap-static & element-summary endpoints. Retrieved from <https://fantasy.premierleague.com/api/bootstrap-static/>
- [3] Tiangolo, S. (2023). FastAPI – Modern, fast web framework for building APIs with Python 3.9+. Retrieved from <https://fastapi.tiangolo.com/>
- [4] Meta Platforms. (2024). React 18 – A JavaScript library for building user interfaces. Retrieved from <https://react.dev/>
- [5] SQLite Consortium. (2024). SQLite – A C-language library that implements a small, fast, self-contained SQL database engine. Retrieved from <https://sqlite.org/>
- [6] GitHub, Inc. (2024). GitHub Actions – Automate, customise, and execute software development workflows. Retrieved from <https://github.com/features/actions>
- [7] Draw.io / diagrams.net. (2024). Free online diagram software. Retrieved from <https://www.diagrams.net/>
- [8] ISO/IEC/IEEE. (2018). ISO/IEC/IEEE 12207:2017 – Systems and software engineering – Software life cycle processes. ISO.
- [9] ISO/IEC. (2011). ISO/IEC 25010:2011 – Systems and software engineering – System and software quality models. ISO.

1.5 Overview

This document presents the complete design and architecture of the Fantasy Tactics League, structured to align with the BIL 482 Software Quality checklist. Each section maps directly to a required control point.

Section	Title	Contents Summary
1.1	Project Purpose	Project description, motivation, and primary technical objectives.
1.2	Project Scope	Scope table listing included and excluded features.
1.3	Definitions, Acronyms & Abbrev.	Three separate reference tables for terms, acronyms, and abbreviations.
1.4	References	APA-formatted numbered bibliography of all external sources.
1.5	Overview	This section; document structure guide.
2.1	Product Perspective	System context, external integrations, and architectural positioning.
2.2	Product Functions	Functional requirements FR-1 to FR-5 with measurable acceptance criteria.
2.3	User Characteristics	User categories, demographics, and skill-level profiles.
2.4	Constraints	Technical, schedule, and architectural constraints.
2.5	Assumptions & Dependencies	Technology assumptions and external service dependencies.
3.1.1	User Interfaces	UI design, component descriptions, and mockup references.
3.1.2	Hardware Interfaces	Supported platforms and runtime environment requirements.
3.1.3	Software Interfaces	External APIs, frameworks, databases, and tooling.
3.1.4	Communication Interfaces	Event Bus Pub/Sub protocol and REST API contracts.
3.2	Functional Requirements	FR list with use-case traceability matrix.
3.3	Performance Requirements	Latency, throughput, and coverage targets.
3.4	Design Constraints	Mandatory patterns, technology stack, and budget constraints.
3.5.1	Reliability	Graceful degradation, null-safe scoring, transaction rollback.
3.5.2	Availability	In-memory caching, API fallback, and uptime considerations.
3.5.3	Security	SQL injection prevention, input sanitisation, OAuth2 roadmap.
3.5.4	Maintainability	CF=0 coupling, SOLID principles, modular event-driven design.

Section	Title	Contents Summary
3.5.5	Portability	Zero-Config deployment, cross-platform, SQLite → PostgreSQL path.
3.6	Other Requirements	Usability, accessibility, and additional non-categorised needs.
4.2	Data Collection	ETL pipeline; FPL API extraction, transformation, and SQLite load.
4.5	Data Limitations & Assumptions	Null handling, API rate limits, and concurrency caveats.
6.1	Software Quality Assurance Plan	Quality metrics table with targets, methods, and responsibilities.
6.2	Verification & Validation	34 test cases, bug records, pass/fail results, and traceability.
6.3	Config. & Change Management	GitHub-based change workflow and version control strategy.
6.4	Risk & Defect Management	Risk register, mitigation strategies, and resolved bug table.
6.5	Product Evaluation & Acceptance	Measurable acceptance criteria mapped to quality factors.
7.3	Risk Analysis & Mitigation	Architectural risk decisions and recommended improvements.
8.1	Target Audience	User segments with demographic and skill-level profiles.
8.2	User Stories	Use-case-derived user stories with requirement mapping.
9.1	Societal Benefits	Educational, technical, and strategic-thinking contributions.
9.2	Economic Constraints	Open-source stack, 10 PH budget rationale.
9.3	Legal & Ethical Compliance	API ToS, data privacy, scope exclusions.

2.1 Product Perspective

Fantasy Tactics League is an Event-Driven Modular Monolith web application. It sits at the centre of three external integrations: the official FPL API (data source), an SQLite database (persistence), and a React browser client (presentation). None of these integrations are tightly coupled to the business logic; instead, they communicate through well-defined REST contracts and an internal Event Bus.

Unlike a standard CRUD application, the system is specifically designed to demonstrate Clean Architecture and GoF Design Patterns in a realistic domain. This dual purpose — functional fantasy football platform and academic architecture showcase — distinguishes it from generic gaming applications.

Architectural Positioning

The Modular Monolith was chosen over Microservices based on a strategic Effort vs. Impact analysis. By enforcing zero direct imports between use-case modules ($CF = 0$) and communicating exclusively via the Event Bus, the system retains full decomposition potential: any module can be extracted into an independent microservice in the future with minimal refactoring.

The system architecture is illustrated in Figure 1.1 (Top-Level Architecture Diagram) and Figure 3.1 (High-Level Modular Monolith Architecture) of the original design document, showing the decoupled interaction between the External Data Layer, the SQLite Persistence Layer, and the API-driven Frontend.

2.2 Product Functions

The table below lists each functional requirement alongside its measurable acceptance criterion, metric, and target value, in accordance with ISO/IEC/IEEE 12207.

Req. ID	Description	Acceptance Criterion	Metric	Target Value
FR-1	Squad Management: Users shall build a 15-player squad (2 GKs, 5 DEFs, 5 MIDs, 3 FWDs) within a £100.0m budget.	System blocks saving if any rule is violated; returns descriptive HTTP 400 error.	Validation pass rate under correct inputs	100%
FR-2	Rule Validation: Max 3 players per club; positional counts enforced.	All 4 strategy classes independently reject invalid squads.	Strategy unit test pass rate	100% (TC01–TC08)
FR-3	Automated Scoring: Calculate individual player points from FPL match stats.	Scoring engine produces correct points for all position/role combinations.	Scoring unit test pass rate	100% (TC09–TC22)
FR-4	Tactical Roles: Captain earns $\times 2$ points; bench players earn $\times 0$.	ModifierStrategy decorator correctly applies multipliers.	Decorator test pass rate	100% (TC21–TC22)
FR-5	Global Ranking: Teams ranked by total points; budget as tie-breaker.	Leaderboard returns sorted list; Singleton returns same instance.	Leaderboard test pass rate	100% (TC23–TC26)

2.3 User Characteristics

User Category	Age Range	Technical Level	Domain Knowledge	Primary Goal
Football Fans	18–45	Basic (web user)	High – Premier League follower	Build competitive fantasy squads and track weekly points.
Strategic Managers	20–40	Intermediate	High – familiar with FPL mechanics	Optimise budget allocation and captain selection for maximum points.
Academic Reviewers	22–55	Advanced (dev/researcher)	Moderate – software design focus	Evaluate correct application of design patterns and architectural decisions.
System Administrator	25–45	Advanced	Low – football domain	Trigger gameweek data fetches and monitor ETL pipeline health.

All users interact with the system through a modern web browser. No installation is required on the client side. The administrator role requires command-line access to invoke the POST `/admin/fetch-gameweek/{id}` endpoint.

2.4 Constraints

Technical Constraints

- Budget Limit: Virtual squad cost must not exceed £100.0m.
- Squad Composition: Exactly 15 players — 2 GK, 5 DEF, 5 MID, 3 FWD.
- Club Limit: Maximum 3 players from any single Premier League club.
- API Rate Limit: FPL API enforces request throttling; ETL scripts use caching to avoid violations.
- Technology Stack: Python 3.9+, FastAPI, React 18, SQLite (embedded).
- Design Pattern Mandate: At least five GoF design patterns must be implemented and demonstrable.

Schedule Constraint

- All deliverables finalised and submitted by April 2026.
- Development budget: 10 Person-Hours (PH) — the primary reason Microservices architecture was excluded.

Architectural Constraints

- Zero direct imports between use-case folders (CF = 0).
- All cross-module communication must pass through the Event Bus.
- Concurrency: The current GlobalLeaderboard Singleton does not implement thread-safe locking — acceptable for academic scope but noted as a known limitation.

2.5 Assumptions and Dependencies

Assumptions

- The official FPL API is accessible and returns data in valid JSON format.
- The server running the application has an active internet connection during data fetch operations.
- Users access the application through a modern web browser (Chrome 100+, Firefox 100+, Safari 15+).
- Users possess basic familiarity with fantasy football concepts (squad selection, captaincy, budget).
- The FPL API schema does not change significantly during the project lifecycle.
- Concurrent user count is low (academic demo context); thread-safety is not critical for the current scope.

Technology Dependencies

Dependency	Version	Role	Risk if Unavailable
Python	3.9+	Backend runtime	System cannot start.
FastAPI	0.100+	REST API framework	API endpoints unavailable.
React	18	Frontend SPA framework	UI inaccessible; API still functional.
SQLite	3.x	Embedded database	No data persistence.
FPL API	Public	Player & match data source	ETL fails; cached data used instead.
pytest	7+	Test framework	Automated tests cannot run.

3.1.1 User Interfaces

The frontend is a Single Page Application (SPA) built with React 18 and Tailwind CSS, following a Glassmorphism design philosophy with a dark-slate palette (#0f172a).

UI Component List

Component	Location	Key Features
Main Dashboard & Analytics	/ (root route)	Top-performing players by goals, assists, points. Data visualisation cards. (Figure 6.1)
Interactive Pitch – Team Builder	/team	Drag-and-drop football pitch. Real-time budget counter. Position slot validation. (Figure 6.2)
Global Standings / Leaderboard	/leaderboard	Live rankings from Singleton cache. Rank, team name, points, budget. (Figure 6.3)
Player Search & Filtering	/players	Filter 800+ players by position (GK/DEF/MID/FWD) and sort by cost or points. (Figure 6.4)

UI Requirements

- UI-1: Budget counter must update in real-time as players are added or removed.
- UI-2: Validation errors must be displayed as toast notifications within 200 ms of submission.
- UI-3: Player list must support filtering by all four positions simultaneously.
- UI-4: Leaderboard must refresh automatically when the backend broadcasts a score update.
- UI-5: The application must be fully usable on 1280×720 resolution and above.

3.1.2 Hardware Interfaces

Fantasy Tactics League is a purely software-based web application and does not interface with physical hardware components. The following table documents the execution environment requirements.

Platform	Minimum Spec	Tested On	Notes
Server OS	Linux / macOS / Windows with Python 3.9+	Ubuntu 24, macOS 14, Windows 11	Zero-Config deployment; no native hardware drivers required.
Client OS	Any OS supporting a modern browser	Chrome 124, Firefox 126	SPA served as static assets; no client-side installation.
RAM	512 MB minimum	2–8 GB recommended	Singleton cache is in-memory; minimal footprint for academic scale.
Storage	50 MB for SQLite DB + application	SSD recommended	DB grows with gameweek data; negligible at project scale.
Network	Outbound HTTPS to FPL API during ETL	Standard broadband	Cached data used if API is unreachable.

3.1.3 Software Interfaces

Interface	Type	Protocol / Format	Endpoint / Location	Input	Output
FPL API – bootstrap-static	External REST API	HTTPS / JSON	https://fantasy.premierleague.com/api/bootstrap-static/	HTTP GET (no params)	JSON: player list, costs, positions, current GW
FPL API – element-summary	External REST API	HTTPS / JSON	/api/element-summary/{player_id}/	player_id (int)	JSON: historical stats per gameweek
FastAPI Backend	Internal REST API	HTTP / JSON	localhost:8000	JSON request body or query params	JSON response + HTTP status
SQLite Database	Embedded RDBMS	SQL / File I/O	fantasy.db (local file)	SQL queries via sqlite3 module	Result sets (rows/dicts)
React Frontend	Browser SPA	HTTP / JSON	Served by Uvicorn on port 8000	User interactions (click, drag)	Rendered HTML/CSS
GitHub	VCS / CI	HTTPS / Git	github.com/group-repo	git push, PR, issue creation	Version history, CI results

3.1.4 Communication Interfaces

External Communication (Client ↔ Server)

The React frontend communicates with the FastAPI backend exclusively via asynchronous HTTP/1.1 REST calls. All payloads are JSON-encoded (Content-Type: application/json). Key endpoints:

Method	Endpoint	Description	Request Body / Params	Success Response
GET	/players	List all players	?team=, ?position=, ?sort_by=	200 OK + JSON array
GET	/stats/latest	Current gameweek stats	–	200 OK + JSON
GET	/player/{id}/history	Player history by ID	id (path)	200 OK + JSON; 404 if unknown
POST	/team	Create and validate squad	JSON: {player_ids: [...]}	200 OK + team_id; 400 if invalid
GET	/leaderboard	Get sorted rankings	–	200 OK + JSON array
POST	/admin/fetch-gameweek/{gw_id}	Trigger ETL for new GW	gw_id (path)	202 Accepted (async)

Internal Communication (Event Bus – Pub/Sub)

Modules communicate internally through a Singleton EventBus using a publish/subscribe model. No direct imports between use-case modules are permitted (CF = 0).

Event Name	Publisher	Subscriber(s)	Triggered When	Payload
GAMEWEEK_DATA_FETCHED	GameweekDataFetcher (UC4)	TeamScoreUpdaterObserver	New GW data successfully loaded into DB.	{ gameweek_id: int }
TEAM_SCORES_UPDATED	TeamScoreUpdaterObserver	GlobalLeaderboard (UC3)	All team scores recalculated for a GW.	{ team_id, name, score, budget }

Event Name	Publisher	Subscriber(s)	Triggered When	Payload
LEADERBOARD_REFRESHED	GlobalLeaderboard (UC3)	ConsoleNotifier, UIEventNotifier	Leaderboard cache updated.	{ sorted_board: [...] }

3.2 Functional Requirements

Requirement Definitions

- FR-1 – Squad Management: The system shall allow a user to build and save a 15-player squad composed of real Premier League players, subject to positional and budget rules.
- FR-2 – Rule Validation: The system shall automatically enforce four validation rules: budget ($\leq$ £100.0m), squad size ($=$ 15), positional distribution (2/5/5/3), and club limit ($\leq$ 3 per club).
- FR-3 – Automated Scoring: The system shall calculate each player's weekly fantasy points based on real match statistics fetched from the FPL API.
- FR-4 – Tactical Roles: The system shall support captain designation (score $\times 2$) and bench player designation (score $\times 0$) via the Decorator Pattern.
- FR-5 – Global Ranking: The system shall maintain a leaderboard sorted by total points descending; budget remaining used as tie-breaker.

Requirement–Use Case Traceability Matrix

Req. ID	Use Case	Related Design Pattern	Related Tests	Status
FR-1	UC-1 Team Builder	Strategy (BudgetValidation, SquadSizeValidation)	TC01–TC03	Implemented
FR-2	UC-1 Team Builder	Strategy (PositionValidation, TeamLimitValidation)	TC04–TC08	Implemented
FR-3	UC-2 Scoring Engine	Strategy (GoalkeeperScoringStrategy, etc.)	TC09–TC20	Implemented
FR-4	UC-2 Scoring Engine	Decorator (ModifierStrategy)	TC21–TC22	Implemented
FR-5	UC-3 Global Leaderboard	Singleton + Observer	TC23–TC26	Implemented

3.3 Performance Requirements

ID	Requirement	Metric	Target Value	Measurement Method	Achieved
NFR-1	API response time for leaderboard endpoint	Latency (ms)	< 200 ms	Load test on GET /leaderboard	< 50 ms (Singleton cache)
NFR-2	Player list retrieval (800+ items)	Latency (ms)	< 1000 ms	Timed GET /players	Achieved
NFR-3	Team save and validation	Latency (ms)	< 200 ms	Timed POST /team	Achieved
NFR-4	Weekly point calculation for all teams	Processing time (s)	< 10 s	Background task timer	Achieved
NFR-5	Scoring engine code coverage	Coverage (%)	> 85%	PyTest coverage report	Target
NFR-6	Data loss during bulk GW insert under failure	Data loss (%)	0%	Try-Except-Rollback stress test	0% (rollback verified)

3.4 Design Constraints

Mandatory Pattern Constraint

At least five GoF Design Patterns must be implemented and demonstrable in source code:

Pattern	Category	Applied In	Justification
Factory Method	Creational	PlayerFactory → Goalkeeper, Defender, Midfielder, Forward	Decouples player instantiation from business logic; Open-Closed Principle.
Strategy	Behavioral	ValidationStrategy, ScoringStrategy (4 position variants)	Interchangeable rule sets without modifying the host class.
Observer	Behavioral	EventBus → ConsoleNotifier, UIEventNotifier, TeamScoreUpdater	Loose coupling between modules; zero direct imports (CF=0).
Decorator	Structural	ModifierStrategy wrapping ScoringStrategy	Dynamic multipliers (captain ×2, bench ×0) without class explosion.
Singleton	Creational	GlobalLeaderboard, EventBus	Single source of truth; prevents data inconsistency.

Technology Stack Constraint

- Backend: Python 3.9+ with FastAPI (async REST).
- Frontend: React 18 SPA with Tailwind CSS.
- Database: SQLite (embedded, Zero-Config).
- Version Control: GitHub.

Schedule & Budget Constraint

- Submission deadline: April 2026.
- Development effort: 10 Person-Hours — microservices architecture explicitly excluded.

3.5.1 Reliability

Graceful Degradation

The scoring subsystem uses `.get()` operations with a default value of 0 for all player statistics. This prevents runtime crashes if the FPL API returns incomplete data. If a player has no stats entry for a gameweek, the `TeamPointCalculator` awards 0 points by default (verified by TC09 – GK 0 minutes = 0 pts).

Transaction Integrity

All bulk data loading from the FPL API is wrapped in a database transaction block. If any error occurs during the batch insert, the system executes a Rollback, leaving the database in its last consistent state. This mechanism directly resolves BUG03 (schema not created on first run) and prevents partial data corruption.

Null-Safe Scoring (BUG01 Resolution)

A pre-processing loop in `calculate_team_score()` replaces all `None` values with 0 before scoring logic executes, eliminating the `TypeError: '>=' not supported for NoneType` that was identified during testing.

Reliability Metrics

Criterion	Target	Achieved
Crash on null/missing stat input	0 crashes	0 crashes (post BUG01 fix)
Data loss on bulk insert failure	0% loss	0% (transaction rollback)
Deterministic scoring (same input → same output)	100%	100% (stateless strategy functions)
Test pass rate	100%	100% (34/34 test cases passed)

3.5.2 Availability

Fantasy Tactics League is an academic prototype deployed locally. Formal uptime SLAs are not applicable; however, the following mechanisms improve operational availability.

In-Memory Singleton Cache

The GlobalLeaderboard Singleton stores all ranking data in RAM. Even if the SQLite database is temporarily unavailable, the leaderboard endpoint continues to serve cached data. This reduces the effective downtime for the most-accessed read endpoint to near-zero under normal operating conditions.

API Fallback

If the FPL API is unreachable during an ETL run, the `fpl_fetcher.py` script logs a `requests.HTTPError` and the system continues to operate using the most recently cached gameweek data from SQLite. No user-facing functionality is disrupted.

Known Limitation

The GlobalLeaderboard Singleton does not implement thread-safe locking (e.g., `threading.Lock()`). Under simultaneous concurrent score updates, a Race Condition could temporarily produce stale rankings. This is accepted as an academic-scope trade-off and is logged as a future improvement (see Section 7.3).

3.5.3 Security

Implemented Security Measures

- SQL Injection Prevention: The sort_by query parameter is sanitised; unsupported values are silently defaulted to total_points rather than passed to the database. This was explicitly verified by TC30 (sort param injection blocked — PASS).
- Input Validation: All player IDs submitted via POST /team are validated against the database before any processing. Non-existent IDs return HTTP 400 (TC34 – PASS).
- Error Isolation: Exception handlers at the FastAPI router level prevent internal stack traces from being exposed to clients.

Current Limitations & Roadmap

Security Area	Current Status	Planned Improvement
Authentication	None (open access)	OAuth2 / JWT token-based auth (Section 8.2 – Future Work)
Authorisation	None	Role-based access control (Admin vs User)
Transport Encryption	HTTP (localhost dev only)	HTTPS / TLS for production deployment
Data Privacy	No PII collected	GDPR compliance on user data if multi-user deployed
Rate Limiting	Not implemented	API gateway rate limiting for public deployment

3.5.4 Maintainability

Quantitative Maintainability Metrics

Metric	Definition	Target	Achieved
Coupling Factor (CF)	Direct imports between use-case folders	0	0 – all cross-module calls via Event Bus
Logic Separation	% of modules with single domain responsibility	100%	100% – each module owns exactly one UC
Design Pattern Usage	Number of GoF patterns correctly applied	≥ 5	5 (Factory, Strategy, Observer, Decorator, Singleton)
Type Hint Coverage	Python functions with type annotations	> 90%	All abstractions use type hints (e.g., <code>→ Tuple[bool, str]</code>)

SOLID Principles Applied

- Single Responsibility: Each module (team.py, strategies.py, observers.py) has exactly one reason to change.
- Open-Closed: New validation rules and player positions are added by creating new Strategy/Factory subclasses, not by editing existing code.
- Liskov Substitution: All ScoringStrategy subclasses are interchangeable via the common `calculate_score(stats: dict) → int` interface.
- Interface Segregation: strategies.py exposes only `validate(team) → Tuple[bool, str]` to external callers.
- Dependency Inversion: TeamPointCalculator depends on the ScoringStrategy abstraction, not on concrete position classes.

3.5.5 Portability

Portability Dimension	Specification	Status
Operating System	Windows 10+, macOS 12+, Linux (Ubuntu 20+)	Verified
Browser Compatibility	Chrome 100+, Firefox 100+, Safari 15+, Edge 100+	Verified
Python Runtime	Python 3.9+ (no OS-specific APIs used)	Zero-Config
Database	SQLite embedded — no server installation required	Zero-Config
Database Migration	SQLAlchemy ORM compatible → PostgreSQL migration < 1 hour	Roadmap
Mobile	Responsive design via Tailwind CSS breakpoints	Partial (not natively tested)

Deployment Steps (Zero-Config)

- Step 1: `python database_setup.py` → Initialises SQLite schema.
- Step 2: `python fpl_fetcher.py` → Fetches and processes live FPL data.
- Step 3: `python -m uvicorn api:app --port 8000` → Launches the web server.

3.6 Other Requirements

Usability Requirements

- The Team Builder and Dashboard must be navigable by a football fan with no technical background within 5 minutes of first use.
- Budget violations must be visually highlighted (e.g., red negative balance) to prevent user errors before submission.
- Player search and filtering (by name, position, club) must return results without page reload.

Accessibility

- All interactive elements must be keyboard-navigable.
- Colour contrast ratios must meet WCAG 2.1 AA standard (4.5:1 for normal text).

Internationalisation

- The current version supports English only. Locale support is deferred to a future release.

4.2 Data Collection

All player and match data is sourced from the official FPL API via the ETL pipeline implemented in `fpl_fetcher.py`.

ETL Stage	Action	Technical Implementation	Output
Extract	HTTP GET requests to FPL API endpoints.	<code>requests.Session()</code> with custom User-Agent headers.	Raw JSON objects.
Transform	Map API field names to internal schema; handle null/missing values.	<code>.get()</code> with default 0; positional mapping (1=GK, 2=DEF...).	Clean Python dicts ready for DB insert.
Load	Batch-insert into SQLite stats and players tables.	INSERT OR REPLACE within a single transaction block.	Populated SQLite DB; rollback on any error.

Data Update Trigger

An administrator calls `POST /admin/fetch-gameweek/{gameweek_id}`. FastAPI delegates the heavy DB operation to a `BackgroundTask`, returning an immediate HTTP 202 Accepted to avoid blocking the main API thread. The ETL completes asynchronously and publishes `GAMEWEEK_DATA_FETCHED` via the Event Bus upon success.

4.5 Data Limitations and Assumptions

Limitation / Assumption	Description	Mitigation
Null / Missing Stats	Players who did not play in a GW may have NULL values in the stats table.	.get(field, 0) default in all scoring strategies; pre-processing loop replaces None with 0 (BUG01 fix).
API Rate Limiting	FPL API enforces throttling; bulk fetches may be delayed.	Caching; ETL runs as background task; existing data remains live.
Full-Dataset Fetch	Current ETL copies the entire player dataset each run (no delta/diff optimisation).	Accepted for academic scale; delta-based fetching recommended for production (Review Report §13).
Thread Safety	GlobalLeaderboard Singleton does not use threading.Lock(); concurrent writes may cause stale reads.	Accepted for single-threaded academic demo; threading.Lock() recommended for production.
Data Freshness	Stats reflect the last successful ETL run; live match data is not streamed.	Out of scope by design; real-time streaming excluded from project scope.
API Schema Stability	System assumes FPL API field names remain stable throughout the project lifecycle.	If schema changes, only the Transform stage of ETL requires updates.

6.1 Software Quality Assurance Plan (SQAP)

Purpose and Scope

This SQAP defines the quality objectives, activities, responsibilities, and success criteria for Fantasy Tactics League. It applies to all four use-case modules (UC1–UC4) and their associated design pattern implementations.

Quality Objectives and Metrics

Quality Factor	Criterion	Metric	Target	Responsible	Verification Method
Correctness	All test cases pass	Test pass rate	100%	All members	pytest – 34 test cases
Maintainability	Low coupling between modules	Coupling Factor (CF)	0	All members	Static dependency scan
Modularity	Single responsibility per module	Logic separation	100%	Architecture owner	Architectural review
Efficiency	Fast leaderboard response	API latency (ms)	< 200 ms	Backend developer	Load test on /leaderboard
Integrity	No data loss under failure	Data loss (%)	0%	Backend developer	Rollback stress test
Testability	High scoring engine coverage	Code coverage (%)	> 85%	Test responsible	PyTest coverage report
Security	SQL injection blocked	Vulnerabilities	0	API & Data manager	TC30 injection test

SQA Activities

- Code Review: All pull requests reviewed by at least one other team member before merge.
- Unit Testing: pytest-based tests for all business logic (strategies.py, calculator.py, leaderboard.py).
- Integration Testing: Live API tests against a locally hosted Uvicorn server with real Gameweek 31 data.
- Static Analysis: Manual dependency scan to verify CF = 0 (no cross-use-case imports).
- Bug Tracking: All defects logged as GitHub Issues with root cause and fix documented.

Roles and Responsibilities

Role	Member(s)	QA Responsibility
Backend Developer	Serhan Tezcan	Core logic correctness, scoring tests, transaction tests.
API & Data Manager	Doruk Dolaşık	Integration tests, API endpoint tests, SQL injection test.
Testing & Integration Lead	Serhan Tezcan / Doruk	Test suite design, pytest execution, bug triage.
Architecture Reviewer	Muhammet Emir Dizdar	Design pattern correctness, CF = 0 verification.

6.2 Verification and Validation (V&V)

Test Summary

Category	Test Cases	Passed	Failed
UC1 – Team Builder	TC01–TC08 (8 cases)	8	0
UC2 – Points Calculator	TC09–TC22 (14 cases)	14	0
UC3 – Global Leaderboard	TC23–TC26 (4 cases)	4	0
UC4 – API Integration	TC27–TC34 (8 cases)	8	0
TOTAL	34 cases	34	0

Bug Fix Record

Bug ID	Description	Root Cause	Fix Applied
BUG01	TypeError on API startup (NoneType >= comparison)	NULL DB values passed to arithmetic in calculate_team_score()	Pre-processing loop replaces None with 0 before scoring.
BUG02	fpl_fetcher.py saved to wrong database	Hardcoded sqlite3.connect('fantasy.db') instead of shared DB module.	Updated save_to_db() to use core.database.get_db_connection().
BUG03	DB schema not created on first run	create_schema() not called before API startup.	Added create_schema() inside @app.on_event('startup').

Test Environment

- Framework: pytest (unit tests) + requests (live API integration tests).
- Server: Locally hosted Uvicorn server on port 8000.
- Database: SQLite (fantasy.db) pre-populated with real Gameweek 31 data.
- Platform: Ubuntu 24 / Python 3.11.

6.3 Configuration and Change Management

Version Control Strategy

- Primary tool: GitHub (git).
- Branching: feature branches per use case; merged to main via Pull Request.
- Contribution tracking: GitHub commit history and issue tracker.

Change Management Process

Any change to requirements or architecture follows this four-step process:

- Step 1 – Discuss: Change is raised and discussed in the team's WhatsApp/Discord channel.
- Step 2 – Document: Change is logged as a GitHub Issue with title, description, and scope impact.
- Step 3 – Evaluate: Impact on timeline, architecture, and test suite is assessed collectively.
- Step 4 – Approve & Implement: Change is approved by all members before a branch is created.

Version Log

Version	Description	Date
v1.x	Core OOP models; Factory Pattern for player instantiation.	March 2026
v2.x	Strategy Pattern for validation; Observer Pattern for real-time event logging.	March 2026
v3.0	Full-stack web architecture: FastAPI backend + React SPA frontend.	April 2026

Future CI/CD Roadmap

GitHub Actions will be configured to automatically run pytest on every pull request and deploy to a cloud environment on merge to main (deferred to post-submission phase).

6.4 Risk and Defect Management

Project Risk Register

Risk ID	Risk	Probability	Impact	Mitigation Strategy
R-01	FPL API rate limiting or downtime during ETL.	Medium	Medium	Use cached/mock data; ETL runs as background task.
R-02	Integration difficulties between React frontend and FastAPI.	Low	High	Early API contract definition; Postman tests before frontend integration.
R-03	Time management — underestimating module complexity.	Medium	High	Weekly progress meetings; 10 PH budget drives monolith decision.
R-04	Team coordination — duplicated or conflicting code.	Low	Medium	Clear role assignment; GitHub branch-per-feature workflow.
R-05	Thread-safety issue in GlobalLeaderboard under load.	Low (academic scale)	Low	Accepted; <code>threading.Lock()</code> recommended for production (§7.3).

Resolved Defects

Three defects were identified and resolved during the testing phase. Full details are in Section 6.2 (Bug Fix Record). All defects were resolved before final submission; test pass rate remained 100%.

6.5 Product Evaluation and Acceptance

The project will be considered successfully completed when all of the following measurable acceptance criteria are met:

Acceptance Criterion	Metric	Target Value	Evidence
All functional requirements implemented	FR pass rate	100%	TC01–TC34 all PASS
All four use cases fully operational	UC completion	4 / 4	UC1–UC4 test suites pass
Five GoF design patterns correctly applied	Pattern count	≥ 5	Design Pattern chapter + class diagrams
Zero coupling between use-case modules	Coupling Factor	CF = 0	Static dependency scan
API leaderboard response time	Latency	< 200 ms	Load test result: < 50 ms
No data loss under bulk insert failure	Data loss	0%	Rollback stress test PASS
SQL injection prevention	Vulnerabilities	0	TC30 PASS
All documentation and GitHub contributions submitted	Deliverable checklist	100%	GitHub repository

7.3 Risk Analysis and Mitigation Strategies

Architectural Risk Decision: Monolith vs. Microservices

The primary architectural risk was choosing between a Modular Monolith and a Microservices architecture. The decision was based on a formal Effort vs. Impact analysis:

- Microservices Risk: Network latency between services, distributed transaction management (Saga pattern), and container orchestration (Kubernetes) would require approximately 3× more development effort than the 10 PH budget allows.
- Monolith Risk: Potential coupling accumulation over time. Mitigated by enforcing CF = 0 and exclusive Event Bus communication.
- Decision: Modular Monolith adopted. The design is 'decomposition-ready' — any module can be extracted to a microservice with minimal refactoring.

Identified Weaknesses and Recommended Improvements

The following weaknesses were identified in the peer Review Report and are acknowledged as future improvements:

Weakness	Current State	Recommended Solution	Priority
Thread-Safety on GlobalLeaderboard	No locking — race condition possible under concurrent writes.	Add <code>threading.Lock()</code> to <code>update_team_score()</code> and <code>get_sorted_leaderboard()</code> methods.	High (for production)
Full-Dataset API Fetch	Entire player dataset re-fetched each ETL run; no delta/diff.	Implement delta-based fetching using FPL API's 'last_updated' timestamp.	Medium
Observer Notification for UI	UI observers partially implemented; no real-time push to frontend.	Complete <code>UIEventNotifierObserver</code> and integrate WebSocket push to React frontend.	Medium
API Rate Limit Handling	No exponential back-off; repeated failures not cached.	Implement retry logic with exponential back-off in <code>fpl_fetcher.py</code> .	Low

8.1 Target Audience

Segment	Description	Age	Technical Skill	Football Knowledge	Primary Need
Football Fans	Premier League followers who enjoy competitive fantasy games.	18–45	Basic	High	Build and manage a competitive squad; track weekly points.
Strategic Managers	Data-driven users who plan transfers based on fixtures and stats.	20–40	Intermediate	High	Optimise budget allocation and maximise captain points.
Academic Reviewers	Instructors and peers evaluating software architecture quality.	22–55	Advanced	Low	Verify correct use of design patterns and SOLID principles.
System Administrator	Technical user responsible for triggering ETL and monitoring health.	25–45	Advanced	Low	Run gameweek data fetches; monitor logs and error rates.

8.2 User Stories

Story ID	User Story	Related FR	Related UC	Acceptance Criterion
US-01	As a football fan, I want to select 15 players within a £100m budget so that I can participate in the league.	FR-1	UC-1	System saves team if all rules pass; returns HTTP 400 with message if any rule fails.
US-02	As a football fan, I want to see my budget update in real time as I add players so that I avoid overspending.	FR-1	UC-1	Budget counter updates within 200 ms of each player selection in the UI.
US-03	As a strategic manager, I want to designate my best player as captain so that his points are doubled this week.	FR-4	UC-2	Captain's score = base_score × 2; verified by TC21.
US-04	As a strategic manager, I want to see the league standings after each gameweek so that I can compare my team against rivals.	FR-5	UC-3	Leaderboard returns sorted list with rank, team name, points, budget in < 200 ms.
US-05	As a football fan, I want to search and filter the player database by position so that I can efficiently build my squad.	FR-1	UC-4	Filter returns only players of selected position; results appear without page reload.
US-06	As a system admin, I want to trigger a new gameweek data fetch so that player statistics are updated after real matches.	FR-3	UC-4	POST /admin/fetch-gameweek/{id} returns 202 Accepted; scores update asynchronously.
US-07	As a football fan, I want the system to block invalid squads (e.g., 4 players from one club) and tell me why it failed.	FR-2	UC-1	System returns descriptive error message identifying which rule was violated.

User Story – Requirement Traceability

User Story	Functional Requirement	Use Case	Test Cases
US-01, US-02, US-07	FR-1, FR-2	UC-1	TC01–TC08
US-03	FR-4	UC-2	TC21–TC22
US-04	FR-5	UC-3	TC23–TC26
US-05, US-06	FR-3	UC-4	TC27–TC34

9.1 Societal Benefits

Educational Value

- The project provides a concrete, end-to-end demonstration of how software design patterns (GoF) and SOLID principles are applied in a real-world-inspired domain. It serves as a reusable learning resource for students studying software architecture.
- The open-source codebase on GitHub is accessible to anyone wishing to study modular monolith architecture, Event-Driven design, or Python/FastAPI/React integration.

Strategic Thinking and Decision-Making

- The platform encourages users to make data-driven decisions (budget allocation, transfer planning, captain selection), developing analytical thinking skills applicable beyond sport.

Technology Literacy

- By interacting with a well-structured web application, non-technical users are exposed to modern software interfaces and REST API-driven data flows, supporting broader digital literacy.

Accessibility

- The Zero-Config deployment model ensures the platform can run on low-cost hardware with a standard internet connection, lowering barriers to access.

9.2 Economic Constraints

Development Budget

- Estimated development effort: 10 Person-Hours (PH) total.
- This budget directly drove the Modular Monolith architectural decision; Microservices would require approximately 3× more effort.

Technology Cost

Tool / Service	Cost	Notes
Python 3.9+	Free (open-source)	No licensing fees; widely supported.
FastAPI	Free (open-source)	MIT licence; no usage restrictions.
React 18	Free (open-source)	Meta open-source; MIT licence.
SQLite	Free (public domain)	No server cost; embedded in Python stdlib.
FPL API	Free (public API)	Official FPL API; no authentication required for read access.
GitHub	Free (academic)	Free private repos for academic accounts.
Tailwind CSS	Free (open-source)	MIT licence.

Total estimated infrastructure cost: £0. The project is designed to operate entirely on free and open-source tools, making it reproducible by any team with a standard development laptop.

9.3 Legal and Ethical Compliance

FPL API Terms of Service

- The system consumes the official Fantasy Premier League API for non-commercial, educational purposes only.
- No data is redistributed publicly; all fetched data is stored locally in a private SQLite database.
- The project does not claim any affiliation with the Premier League or Fantasy Premier League service.

Data Privacy

- The current version collects no personally identifiable information (PII). Team names are user-provided strings with no linkage to real identities.
- If deployed publicly in a multi-user scenario, GDPR compliance (data minimisation, right to erasure, privacy notice) would be required. OAuth2 authentication is planned (Section 8.2).

Intellectual Property

- All source code is original work by Group S.A.Z, developed for BIL 482 coursework.
- Third-party libraries (FastAPI, React, SQLite) are used under their respective open-source licences (MIT, Apache 2.0, Public Domain).

Ethical Scope Exclusions

- Betting, gambling, and real-money features are explicitly excluded from scope (Section 1.2).
- No automated decision-making with significant individual impact is implemented.
- The platform does not collect behavioural data or user tracking beyond what is required for squad management.